Assignment 3

cpe 453 Spring 2026

```
The men sat sipping their tea in silence.  After a while the klutz said,
        "Life is like a bowl of sour cream."
        "Like a bowl of sour cream?" asked the other.  "Why?"
        "How should I know?  What am I, a philosopher?"
```

— /usr/games/fortune

Due by 11:59:59pm, Friday, May 1st.
This assignment is to be done individually.

Program: dine

Using the pthreads library, implement a version of the Dining Philosophers Problem (described on p.66 in Tannenbaum and Woodhull and below).

The Dining Philosophers is a classic interprocess communication problem posed by Dijkstra in 1965: Five philosophers sit around a table. Between each pair of philosophers is a fork, and in the center of the table is a bowl of spaghetti. The life of a philosopher is very simple. It consists of alternate periods of eating and thinking:

To eat a philosopher requires two forks (either the spaghetti is very slippery, or the philosophers are clumsy). The forks can only be picked up one at a time, so he or she must grab one first, then the other. If a neighbor is holding one of the forks, the philosopher must wait until it has been set down. Once a philosopher has a fork, he or she will not relinquish it until after eating.

Upon gaining possession of both forks, a philosopher eats until full, then gets ready to think.

To think a philosopher puts down both forks, one at a time, then drops off into contemplation until hungry at which point the process starts again.

For the purposes of this assignment, a philosopher can be considered as always being in one of three states: `eating`, `thinking`, or `changing`. The first two states are self-explanatory. The third, `changing`, describes a philosopher who has decided to eat, but who has not yet acquired both forks, who has decided to think, but who has not yet set down both forks, or a process transistioning from thinking to terminated.

All philosophers start out in the `changing` state, and start out hungry. That is, they will all try to eat before beginning to think.

Program requirements:

- While Dijkstra always had five philosophers, your program must be parametereized in terms of a constant, `NUM_PHILOSOPHERS`, that determines the number of philosophers. (Default 5) If the number is greater than the number of letters in the alphabet, just continue on up through the ASCII table for labels[1]. Do the same for the fork labels.

[1] That is, use 'A' plus i for philosopher i. This will lead to some weird characters at big parties, but you often meet weird characters at big parties.

To make it easy to change this value at compile time, please define this constant as shown below:

```
#ifndef NUM_PHILOSOPHERS
#define NUM_PHILOSOPHERS 5
#endif
```

- Your program must use POSIX semaphores (see sem_overview(7)) to control individual forks between the philosophers. That is, you may not use the approach shown in the text.
- Your program must avoid deadlock, and, of course, must ensure that neighboring philosophers are never eating simultaneously.
- You should permit two non-adjacent philosophers to eat at the same time.
- To make the program runs more interesting, have the philosophers linger over their spaghetti for a random amount of time. See *Tricks and Tools* below.
- Each time a philosopher changes state, print a status line—in the format shown below—indicating what has happened. This status line should show the state of each philosopher ("Eat", "Think", or blank for changing), and the forks held by that philosopher.

  State changes include:

  - changing among "eat", "think" and "transition"
  - picking up a fork
  - setting down a fork
- Each status line should show *only one* change.
- You must take as an optional command-line argument an integer indicating how many times each philosopher should go through his or her eat-think cycle before exiting.

  Absent this argument, the value should default to 1.
- To make matters simple, number the forks 0,1,2,3,4, and the philosophers, named A,B,C,D, and E, sit at positions 0.5, 1.5, 2.5, 3.5, and 4.5.

  That is, philosopher A uses forks 0 and 1, philosopher B uses forks 1 and 2, etc.
- you will probably need a binary semaphore around the state printing and updating to ensure that it always prints a consistent state.
- **Write robust code.** Your code should compile cleanly with `-Wall`, and should not crash under **any** error conditions. If it cannot complete its task, it should print an error message and exit gracefully with non-zero exit status.
- **Be careful about synchronization; it's more complicated that it looks**.

Tricks and Tools

Using Pthreads

POSIX Threads (pthreads) is a threading library that can be used on many different platforms. It provides functions for thread creation and termination analogous to `fork(2)`, `exit(3)`, and `wait(2)`.

Be careful when using these functions: most of them pass references to other things. Consider the prototype of `pthread_create()`:

```
int pthread_create(  pthread_t * thread,
                     pthread_attr_t * attr,
                     void * (*start_routine)(void *),
                     void * arg);
```

The first argument is a pointer to a thread descriptor that will be set up by the call to describe the new thread. The second is a pointer to a set of attributes if you want to do something special. For this assignment, `NULL` is sufficient to take the defaults. The third is a function pointer to a function that takes a void pointer and returns a void pointer. The fourth pointer is the parameter to be passed to the function when the thread is started.

Note: void pointers are ANSI C's mechanism for dealing with generic pointers. C does not permit one to dereference a void pointer, so it is necessary to cast it to another pointer type first. Given a void pointer called `ptr`, it can be dereferenced into an int via: `int i = *(int *)ptr;`

The necessary pthreads functions are described in Table 1. Look at the man pages for full descriptions. When using pthreads, be sure to link with the pthreads library by adding "`-lpthread`" to your link line.

Because an example is worth a thousand words (well, 504 in this case), I have included a pthreads implementation of `trivial` in Figure 1.

`sem_overview(7)`	Provides an overview of POSIX semaphores.
`sem_wait(3)`	Decrement a semaphore or block trying. The DOWN() operation.
`sem_post(3)`	Wake a thread blocked on this semaphore if there are any, or increment the semaphore if not. The UP() operation.
`sem_init(3)`	Initialize a pthread semaphore for use. Takes a pointer to the semaphore, a flag determining how the semaphore is to be shared, and an initial value.
`sem_getvalue(3)`	Find out the current value of a semaphore. **Note:** This is *not* safe to rely on for synchronization, but can be valuable for debugging when your program gets stuck.
`sem_destroy(3)`	destroy (deallocate) a pthread semaphore. Takes a pointer to the semaphore.
`pthread_create(3);`	creates a pthread.
`pthread_join(3);`	the pthread equivalent of wait(2).
`pthread_exit(3);`	the pthread equivalent of exit(3).

Table 1: Useful pthreads functions

Some Notes

There are a few peculiarities about pthreads you ought to know:

- Be sure to include `-lpthread` on your link line.

```c
/*
 * trivial_pt.c is a re-make of asgn1's fork-testing program
 * done as as demonstration of pthreads.
 *
 * -PLN
 *
 * Requres the pthread library.  To compile:
 *   gcc -Wall -lpthread -o trivial_pt trivial_pt.c
 *
 * Revision History:
 *
 * $Log: trivial_pt.c,v $
 * Revision 1.1  2003-01-28 12:55:00-08  pnico
 * Initial revision
 *
 *
 *
 */
#include <errno.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

#define NUM_CHILDREN 4

void *child(void *id) {
  /*
   * this function will be executed as the body of each child
   * thread.  It expects a single parameter that is a pointer
   * to an integer, its ID.
   *  The parameter is a void* to comply with the prototype,
   * but we know what's really in there.
   */
  int whoami=*(int*)id;          /* numeric id */

  printf("Child %d (%d):    Hello.\n\n",   whoami, (int) getpid());
  printf("Child %d (%d):    Goodbye.\n\n", whoami, (int) getpid());

  return NULL;                  /* exits the thread with no final message */
}


int main(int argc, char *argv[]) {
  pid_t ppid;
  int i;

  int       id[NUM_CHILDREN];       /* individual identifiers (see below) */
  pthread_t childid[NUM_CHILDREN]; /* thread activations for each child */

  /* initialize the parent process id for later use */
  ppid = getpid();

  /* initialize an array of ID numbers for the children.
   *  It would be tempting to just pass the loop index
   * (like we did with trivial), but a pointer is passed
   * to the new thread, not the argument itself.  Because
   * the loop index will change, the effect is not what
   * one would hope.
   *  So, we give each child its own _independent_ ID
   * in the id array.
   */
  for(i=0;i<NUM_CHILDREN;i++)
    id[i]=i;

  /* Spawn all the children */
  for (i=0;i<NUM_CHILDREN;i++) {
    /* pthread_create() launches a new thread running the function
     * child(), passes a poitner to the argument in id[i], and
     * places a thread identifier in childid[i].
     *
     * A note on C:  below, I write "&childid[i]" to indicate the
     * address of the i-th element of the array child, but I could
     * just as well used pointer arithmetic and written "childid+i".
     */
    int res;
    res = pthread_create(
                         &childid[i], /* where to put the identifier      */
                         NULL,        /* don't set any special properties */
                         child,       /* call the function child()        */
                         (void*) (&id[i])   /* pass the address of id[i]  */
                         );

    if ( res != 0 ) {          /*  there was an error */
      /* report the error condition */
      fprintf(stderr,"Child %i:  %s\n",i,strerror(res));
      exit(EXIT_FAILURE);       /* bail out. */
    }
  }


  /* Say hello */
  printf("Parent  (%d):    Hello.\n\n",(int) ppid);

  /*
   * Now wait for each child thread to finish.
   *   Note: Unlike the original trivial, pthread_join()
   * requires us to name a specific thread to wait for, thus
   * the children will always join in the same order regardless
   * of when they actually terminate.
   */

  for(i = 0 ; i < NUM_CHILDREN ; i++ ) {
    if (pthread_join(childid[i],NULL)) {
      perror("pthread_join");
      exit(EXIT_FAILURE);
    }
    printf("Parent  (%d):    child %d exited.\n\n",
           (int) ppid, i);
  }

  /* Say goodbye */
  printf("Parent  (%d):    Goodbye.\n\n",(int) ppid);

  return 0;                     /* exit successfully */
}
```

Figure 1: A pthreads example: `trivial_pt`

Avoiding Deadlock

The dining philosophers problem is interesting because of the ways in which it can fail. If each philosopher simultaneously picks up the fork to his or her left, there will be no right-hand forks available and no philosopher will be able to get two forks. Since no philosopher will set down a fork in hand, they will all starve to death at the table. In terms of operating systems, we would say that the philosopher processes have deadlocked.

In order to avoid deadlock it is necessary to ensure that such a circular wait cannot happen. It turns out that in the case of the dining philosophers all that is necessary is to break the symmetry. If even philosophers try to pick up their right-hand forks first, and odd philosophers (aren't they all?) try to pick up their left-hand forks first, the problem is avoided.

You can use this information in your solution once you've convinced yourself that it's true.

Randomization

To make the behavior of the philosophers a little more interesting, have them linger a little while eating or thinking. The routine `dawdle()` shown in Figure 2, sleeps for a randomly chosen number of milliseconds up to a second. `dawdle()` must be linked with the realtime library by adding "`-lrt`" to the link line.

Note that `random(3)` will always produce the same pseudorandom sequence unless seeded with `srandom(3)` at the beginning of the run. My published solution uses the number of seconds since the epoch plus the number of microseconds since the last second as the seed. These values are available via `gettimeofday(2)`.

Displaying the status

Every time something changes, it is important to report it so that the person running the program can see what has happened. The format should be as shown below, five columns showing the instantaneous status of all the philosophers. The status should show the philosopher's state (eating, thinking, or changing) and which forks the philosopher is holding.

How you implement this is, of course, up to you, but do remember that more than one philosopher may change state at a time, so you may have to place locks around your status printing routines in addition to the locks on the forks.

Coding Standards and Make

See the pages on coding standards and make on the cpe 453 class web page.

What to turn in

Submit via `handin` to the `asgn3` directory of the `pn-cs453` account:

- your well-documented source files.
- A makefile (called `Makefile`) that will build your program when given the target "`dine`".
- A README file that contains:
  - Your name.
  - Any special instructions for running your program.
  - Any other thing you want me to know while I am grading it.

```
#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <string.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

#ifndef DAWDLEFACTOR
#define DAWDLEFACTOR 1000
#endif

void dawdle() {
  /*
   * sleep for a random amount of time between 0 and DAWDLEFACTOR
   * milliseconds.  This routine is somewhat unreliable, since it
   * doesn't take into account the possiblity that the nanosleep
   * could be interrupted for some legitimate reason.
   */
  struct timespec tv;
  int msec = (int)((((double)random()) / RAND_MAX) * DAWDLEFACTOR);

  tv.tv_sec  = 0;
  tv.tv_nsec = 1000000 * msec;
  if ( -1 == nanosleep(&tv,NULL) ) {
    perror("nanosleep");
  }
}
```

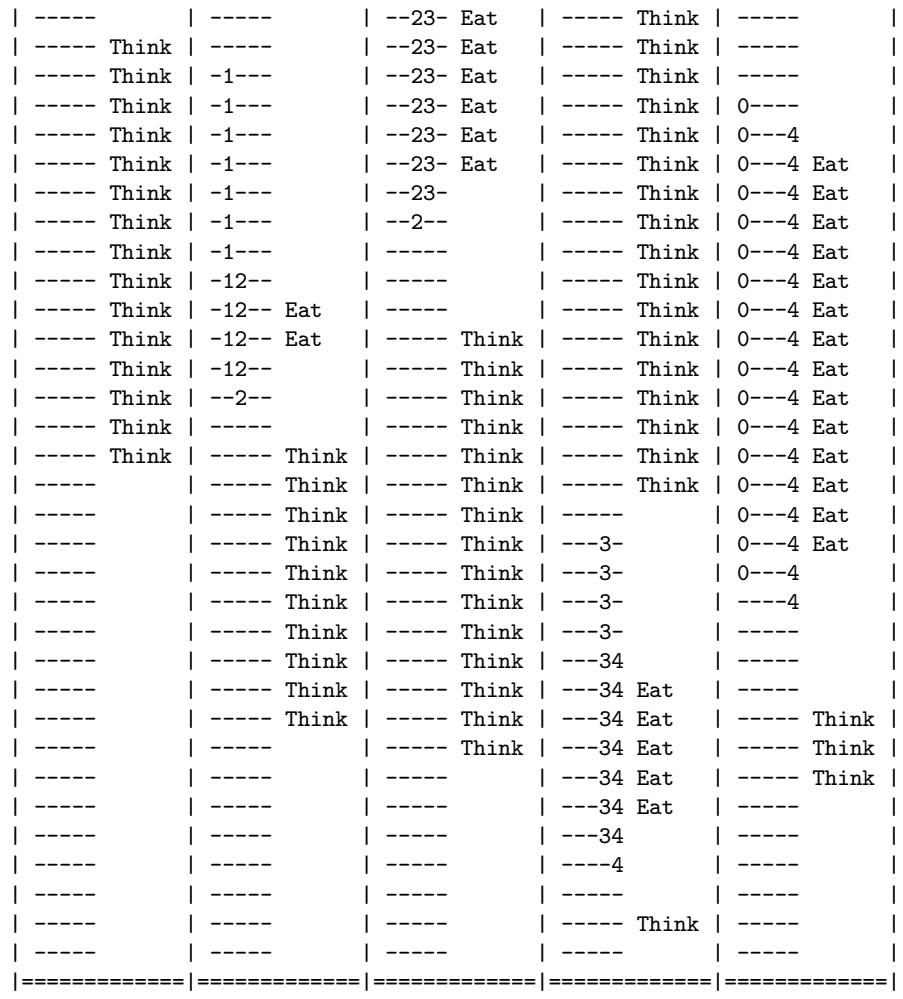
Figure 2: A routine to sleep for a little while

Sample runs

Below are some sample runs of `dine`. I will also place executable versions on the CSL machines in `~pn-cs453/demos` so you can run it yourself.

```
% dine
|=============|=============|=============|=============|=============|
|      A      |      B      |      C      |      D      |      E      |
|=============|=============|=============|=============|=============|
| -----       | -----       | -----       | -----       | -----       |
| -1---       | -----       | -----       | -----       | -----       |
| 01---       | -----       | -----       | -----       | -----       |
| 01--- Eat   | -----       | -----       | -----       | -----       |
| 01--- Eat   | -----       | ---3-       | -----       | -----       |
| 01--- Eat   | -----       | --23-       | -----       | -----       |
| 01--- Eat   | -----       | --23- Eat   | -----       | -----       |
| 01---       | -----       | --23- Eat   | -----       | -----       |
| 0----       | -----       | --23- Eat   | -----       | -----       |
| -----       | -----       | --23- Eat   | -----       | -----       |
| -----       | -1---       | --23- Eat   | -----       | -----       |
| -----       | -1---       | --23- Eat   | -----       | 0----       |
| -----       | -1---       | --23- Eat   | -----       | 0---4       |
| -----       | -1---       | --23- Eat   | -----       | 0---4 Eat   |
| ----- Think | -1---       | --23- Eat   | -----       | 0---4 Eat   |
| ----- Think | -1---       | --23-       | -----       | 0---4 Eat   |
| ----- Think | -1---       | --2--       | -----       | 0---4 Eat   |
| ----- Think | -1---       | -----       | -----       | 0---4 Eat   |
| ----- Think | -1---       | -----       | ---3-       | 0---4 Eat   |
| ----- Think | -12--       | -----       | ---3-       | 0---4 Eat   |
| ----- Think | -12-- Eat   | -----       | ---3-       | 0---4 Eat   |
| ----- Think | -12-- Eat   | ----- Think | ---3-       | 0---4 Eat   |
| ----- Think | -12-- Eat   | ----- Think | ---3-       | 0---4       |
| ----- Think | -12-- Eat   | ----- Think | ---3-       | ----4       |
| ----- Think | -12-- Eat   | ----- Think | ---3-       | -----       |
| ----- Think | -12-- Eat   | ----- Think | ---3-       | ----- Think |
| ----- Think | -12-- Eat   | ----- Think | ---34       | ----- Think |
| ----- Think | -12-- Eat   | ----- Think | ---34 Eat   | ----- Think |
| ----- Think | -12--       | ----- Think | ---34 Eat   | ----- Think |
| ----- Think | --2--       | ----- Think | ---34 Eat   | ----- Think |
| ----- Think | -----       | ----- Think | ---34 Eat   | ----- Think |
| ----- Think | ----- Think | ----- Think | ---34 Eat   | ----- Think |
| ----- Think | ----- Think | ----- Think | ---34 Eat   | -----       |
| -----       | ----- Think | ----- Think | ---34 Eat   | -----       |
| -----       | ----- Think | -----       | ---34 Eat   | -----       |
| -----       | -----       | -----       | ---34 Eat   | -----       |
| -----       | -----       | -----       | ---34       | -----       |
| -----       | -----       | -----       | ----4       | -----       |
| -----       | -----       | -----       | -----       | -----       |
| -----       | -----       | -----       | ----- Think | -----       |
| -----       | -----       | -----       | -----       | -----       |
|=============|=============|=============|=============|=============|
```

```
% dine 2
|=============|=============|=============|=============|=============|
|      A      |      B      |      C      |      D      |      E      |
|=============|=============|=============|=============|=============|
| -----       | -----       | -----       | -----       | -----       |
| -1---       | -----       | -----       | -----       | -----       |
| 01---       | -----       | -----       | -----       | -----       |
| 01--- Eat   | -----       | -----       | -----       | -----       |
| 01--- Eat   | -----       | ---3-       | -----       | -----       |
| 01--- Eat   | -----       | --23-       | -----       | -----       |
| 01--- Eat   | -----       | --23- Eat   | -----       | -----       |
| 01---       | -----       | --23- Eat   | -----       | -----       |
| 0----       | -----       | --23- Eat   | -----       | -----       |
| -----       | -----       | --23- Eat   | -----       | -----       |
| ----- Think | -----       | --23- Eat   | -----       | -----       |
| ----- Think | -1---       | --23- Eat   | -----       | -----       |
| ----- Think | -1---       | --23- Eat   | -----       | 0----       |
| ----- Think | -1---       | --23- Eat   | -----       | 0---4       |
| ----- Think | -1---       | --23- Eat   | -----       | 0---4 Eat   |
| ----- Think | -1---       | --23-       | -----       | 0---4 Eat   |
| ----- Think | -1---       | --2--       | -----       | 0---4 Eat   |
| ----- Think | -1---       | -----       | -----       | 0---4 Eat   |
| ----- Think | -12--       | -----       | -----       | 0---4 Eat   |
| ----- Think | -12-- Eat   | -----       | -----       | 0---4 Eat   |
| ----- Think | -12-- Eat   | ----- Think | -----       | 0---4 Eat   |
| ----- Think | -12-- Eat   | ----- Think | ---3-       | 0---4 Eat   |
| -----       | -12-- Eat   | ----- Think | ---3-       | 0---4 Eat   |
| -----       | -12-- Eat   | -----       | ---3-       | 0---4 Eat   |
| -----       | -12--       | -----       | ---3-       | 0---4 Eat   |
| -----       | --2--       | -----       | ---3-       | 0---4 Eat   |
| -1---       | --2--       | -----       | ---3-       | 0---4 Eat   |
| -1---       | -----       | -----       | ---3-       | 0---4 Eat   |
| -1---       | ----- Think | -----       | ---3-       | 0---4 Eat   |
| -1---       | ----- Think | -----       | ---3-       | 0---4       |
| -1---       | ----- Think | -----       | ---3-       | ----4       |
| 01---       | ----- Think | -----       | ---3-       | ----4       |
| 01--- Eat   | ----- Think | -----       | ---3-       | ----4       |
| 01--- Eat   | ----- Think | -----       | ---3-       | -----       |
| 01--- Eat   | ----- Think | -----       | ---34       | -----       |
| 01--- Eat   | ----- Think | -----       | ---34 Eat   | -----       |
| 01--- Eat   | ----- Think | -----       | ---34 Eat   | ----- Think |
| 01--- Eat   | ----- Think | -----       | ---34 Eat   | -----       |
| 01--- Eat   | ----- Think | -----       | ---34       | -----       |
| 01--- Eat   | ----- Think | -----       | ----4       | -----       |
| 01--- Eat   | ----- Think | ---3-       | ----4       | -----       |
| 01--- Eat   | ----- Think | --23-       | ----4       | -----       |
| 01--- Eat   | ----- Think | --23- Eat   | ----4       | -----       |
| 01--- Eat   | ----- Think | --23- Eat   | -----       | -----       |
| 01--- Eat   | ----- Think | --23- Eat   | ----- Think | -----       |
| 01--- Eat   | -----       | --23- Eat   | ----- Think | -----       |
| 01---       | -----       | --23- Eat   | ----- Think | -----       |
| 0----       | -----       | --23- Eat   | ----- Think | -----       |
```


-----	-----	--23- Eat	----- Think	-----
----- Think	-----	--23- Eat	----- Think	-----
----- Think	-1---	--23- Eat	----- Think	-----
----- Think	-1---	--23- Eat	----- Think	0----
----- Think	-1---	--23- Eat	----- Think	0---4
----- Think	-1---	--23- Eat	----- Think	0---4 Eat
----- Think	-1---	--23-	----- Think	0---4 Eat
----- Think	-1---	--2--	----- Think	0---4 Eat
----- Think	-1---	-----	----- Think	0---4 Eat
----- Think	-12--	-----	----- Think	0---4 Eat
----- Think	-12-- Eat	-----	----- Think	0---4 Eat
----- Think	-12-- Eat	----- Think	----- Think	0---4 Eat
----- Think	-12--	----- Think	----- Think	0---4 Eat
----- Think	--2--	----- Think	----- Think	0---4 Eat
----- Think	-----	----- Think	----- Think	0---4 Eat
----- Think	----- Think	----- Think	----- Think	0---4 Eat
-----	----- Think	----- Think	----- Think	0---4 Eat
-----	----- Think	----- Think	-----	0---4 Eat
-----	----- Think	----- Think	---3-	0---4 Eat
-----	----- Think	----- Think	---3-	0---4
-----	----- Think	----- Think	---3-	----4
-----	----- Think	----- Think	---3-	-----
-----	----- Think	----- Think	---34	-----
-----	----- Think	----- Think	---34 Eat	-----
-----	----- Think	----- Think	---34 Eat	----- Think
-----	-----	----- Think	---34 Eat	----- Think
-----	-----	-----	---34 Eat	----- Think
-----	-----	-----	---34 Eat	-----
-----	-----	-----	---34	-----
-----	-----	-----	----4	-----
-----	-----	-----	----- Think	-----
=============	=============	=============	=============	=============
%